

Kreacijski dizajn paterni i njihova moguća primjena u sistemu Optika Oculis

1. Singleton

Singleton patern osigurava da određena klasa ima samo jednu instancu u cijelom sistemu i da se toj instanci pristupa globalno.

Singleton patern mogao bi se primijeniti na klasu **ObavijestService**.

Sistem šalje obavijesti korisnicima nakon kreiranja narudžbe, promjene statusa narudžbe ili zakazivanja termina pregleda vida. Nije potrebno da se za svaku obavijest kreira novi servis, već cijeli sistem može koristiti jednu zajedničku instancu servisa za obavijesti.

Na taj način se postiže centralizovano upravljanje slanjem obavijesti, smanjuje se nepotrebno kreiranje objekata i osigurava da svi dijelovi sistema koriste isti mehanizam za slanje poruka korisnicima.

2. Factory Method

Factory Method patern omogućava kreiranje objekata bez direktnog pozivanja konkretnih klasa. Odluka o tome koji će se konkretan objekat kreirati prepušta se factory klasama.

U sistemu Oculis ovaj patern se može primijeniti na podsistem za plaćanje. Sistem podržava više načina plaćanja, kao što su kartično plaćanje, gotovinsko plaćanje i PayPal plaćanje. Umjesto da sistem direktno kreira objekte *KarticnoPlacanje*, *GotovinskoPlacanje* ili *PayPalAdapter*, uvodi se apstraktna klasa **PlacanjeFactory** i konkretne factory klase.

Primjeri konkretnih factory klasa su:

KarticnoPlacanjeFactory, *GotovinskoPlacanjeFactory* i *PayPalPlacanjeFactory*.

Svaka od njih kreira odgovarajući objekat koji implementira interfejs **IPlacanje**. Time se smanjuje zavisnost sistema od konkretnih klasa plaćanja. Ako se kasnije doda novi način plaćanja, npr. Apple Pay ili bankovni transfer, potrebno je samo dodati novu klasu plaćanja i novu factory klasu, bez velikih izmjena ostatka sistema.

3. Abstract Factory

Abstract Factory patern omogućava kreiranje familija povezanih objekata bez navođenja njihovih konkretnih klasa.

U sistemu Oculis ovaj patern bi se mogao primijeniti na kreiranje različitih kategorija proizvoda i njihovih pratećih elemenata. Sistem sadrži proizvode kao što su sunčane naočale, dioptrijske naočale, kontaktne sočiva i dodatna oprema.

Na primjer, mogla bi postojati fabrika **DioptrijskeNaocaleFactory** koja kreira povezane objekte za dioptrijske naočale, kao što su proizvod, preporuka pregleda i odgovarajuća dodatna oprema. S druge strane, **KontaktneSočivaFactory** bi kreirala proizvode i preporuke vezane za kontaktne sočiva.

Prednost ovog paterna je u tome što osigurava da se zajedno kreiraju objekti koji logički pripadaju istoj grupi. Tako se smanjuje mogućnost greške, npr. da se preporuka ili dodatna oprema pogrešno poveže sa neodgovarajućom vrstom proizvoda.

4. Builder

Builder patern koristi se kada je potrebno postepeno kreirati složen objekat koji ima veći broj atributa.

U sistemu Oculis ovaj patern se može primijeniti na klasu **Narudzba**. Kreiranje narudžbe uključuje više koraka: izbor korisnika, povezivanje sa korpom, unos adrese isporuke, određivanje statusa narudžbe i izračunavanje ukupne cijene.

Umjesto korištenja velikog konstruktora sa mnogo parametara, uvodi se **INarudzbaBuilder**, konkretna klasa **NarudzbaBuilder** i klasa **NarudzbaDirektor**. Builder omogućava da se narudžba gradi postepeno, npr. prvo se postavi korisnik, zatim korpa, zatim adresa, status i ukupna cijena.

Ovaj pristup čini proces kreiranja narudžbe preglednijim, smanjuje mogućnost grešaka i olakšava kasnije proširenje klase Narudzba novim atributima.

5. Prototype

Prototype patern omogućava kreiranje novih objekata kopiranjem već postojećih objekata, umjesto njihovog kreiranja od početka.

U sistemu ovaj patern se može primijeniti na klasu **TerminPregleda**. Poslovnica može imati veliki broj sličnih termina pregleda vida tokom dana ili sedmice. Umjesto da se svaki termin kreira ručno od početka, sistem može imati osnovni šablon termina koji se klonira, a zatim se promijene samo datum, vrijeme ili poslovnica.

Na primjer, osnovni termin može sadržavati podatke o trajanju pregleda i statusu, a svaki novi termin nastaje kopiranjem tog šablona i izmjenom konkretnih vrijednosti. Time se ubrzava kreiranje većeg broja sličnih termina i smanjuje ponavljanje istih podataka.

